

Custom VPC Setup with Bastion Host and NAT Gateway

Project: Custom VPC Setup with Bastion Host and NAT Gateway

Services: VPC, EC2, NAT Gateway, Bastion Host, Security Groups

Goal: Build a secure private/public subnet architecture with internet access for private instances.

Skills: Routing tables, subnetting, SSH tunneling.

Definition:

The Custom VPC Setup with Bastion Host and NAT Gateway project is designed to create a secure and scalable AWS networking architecture. The setup includes public and private subnets within a custom VPC. Public resources such as a Bastion Host and NAT Gateway are placed in public subnets, while application servers are placed in private subnets for enhanced security. The Bastion Host provides secure SSH access to private instances, and the NAT Gateway enables outbound internet access for private instances without exposing them directly to the internet.

Scope of the Project:

The scope of this project is to design and implement a secure AWS network infrastructure using a custom VPC with public and private subnets. The project ensures secure access management and controlled internet connectivity for private instances.

This project covers:

- Creation of a Custom VPC with public and private subnets.
- Internet Gateway setup for public internet connectivity.
- NAT Gateway configuration for private subnet internet access.
- Bastion Host setup for secure SSH access to private instances.
- Routing table configuration for traffic management.
- Security Group implementation for controlled access.

Methodology:

1. VPC + Public/Private Subnet Architecture – Best Method

- Uses AWS networking services including VPC, Subnets, Internet Gateway, NAT Gateway, and EC2.
- Separates resources into public and private networks for improved security.
- Provides controlled internet access for private instances using NAT Gateway.
- Enables secure administrative access through Bastion Host.
- Production-ready architecture suitable for enterprise deployments.
- Highly scalable and secure cloud networking design.

2. Flat Network Architecture – Alternative Method

- Uses a single public subnet where all instances are directly accessible.
- Easier to configure for small testing environments.
- Lacks proper isolation and security for sensitive resources.
- Increases exposure of application servers to the public internet.
- Not recommended for production workloads.

AWS Services Used in the Project:

1. Amazon VPC (Virtual Private Cloud)

- **Definition:** Amazon VPC enables users to launch AWS resources in a logically isolated virtual network.
- **Purpose:** Provides isolated networking infrastructure for AWS resources.
- **Role:** Hosts public and private subnets, routing tables, and gateways.

2. Amazon EC2 (Elastic Compute Cloud)

- **Definition:** Amazon EC2 provides resizable virtual servers in the cloud.
- **Purpose:** Hosts Bastion Host and private application instances.
- **Role:** Bastion Host provides SSH access; private EC2 instances host applications securely.

3. NAT Gateway

- **Definition:** NAT Gateway allows instances in private subnets to connect to the internet without allowing inbound internet traffic.
- **Purpose:** Enables outbound internet access for private instances.
- **Role:** Allows software updates and package installations from private subnets.

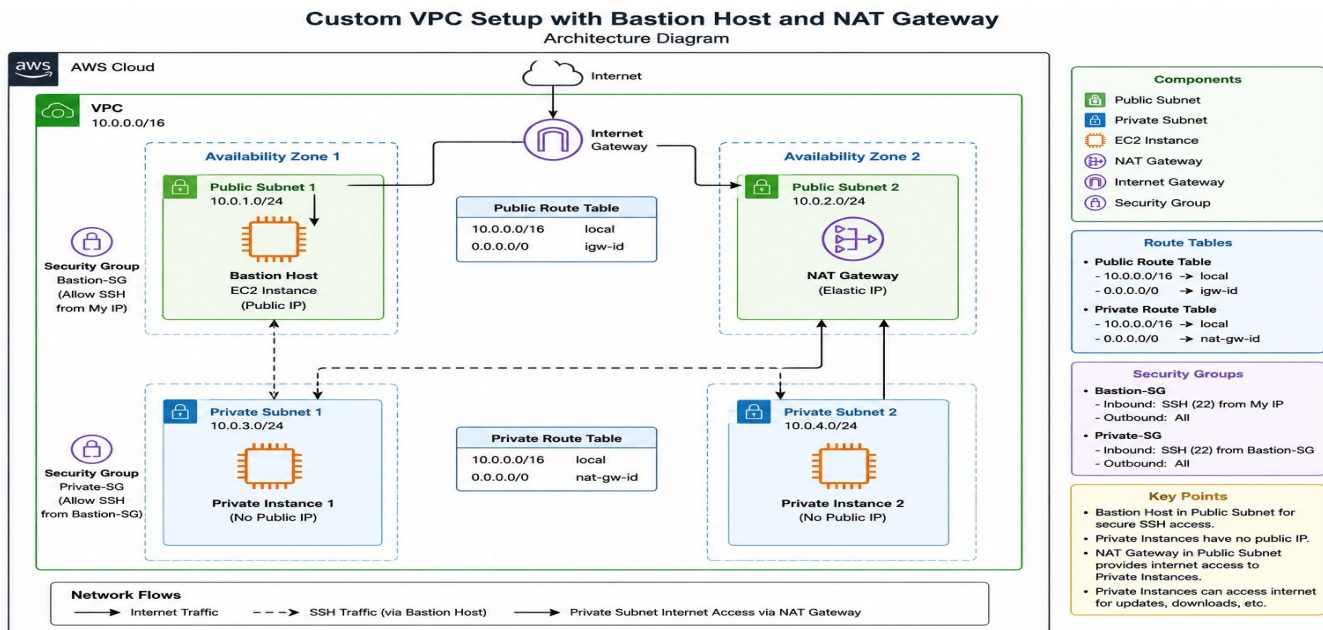
4. Bastion Host

- **Definition:** A Bastion Host is a secure EC2 instance deployed in a public subnet used to access private instances securely.
- **Purpose:** Provides secure SSH connectivity to private instances.
- **Role:** Acts as the entry point for administrators into the private network.

5. Security Groups

- **Definition:** Security Groups act as virtual firewalls controlling inbound and outbound traffic for AWS resources.
- **Purpose:** Restricts and manages network traffic.
- **Role:** Allows SSH access only from trusted sources and controls communication between instances.

Architecture Diagram:



The architecture includes:

- One Custom VPC.
- One Public Subnet.
- One Private Subnet.
- Internet Gateway attached to the VPC.
- NAT Gateway deployed in the Public Subnet.
- Bastion Host deployed in the Public Subnet.
- Private EC2 instance deployed in the Private Subnet.
- Separate Route Tables for public and private subnets.
- Security Groups for traffic control.

Implementation Steps to Do the Project:

Step 1: Create Custom VPC

- Go to AWS Console – VPC – Create VPC.
- Enter VPC Name: Custom-VPC.
- CIDR Block: 10.0.0.0/16.
- Create the VPC.

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

VPC > Your VPCs > vpc-06aabd6bb9dbbe9d0

Filter by VPC

Virtual private cloud

Your VPCs

- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

- Network ACLs
- Security groups

PrivateLink and

vpc-06aabd6bb9dbbe9d0 / Custom-VPC

Details Info

VPC ID vpc-06aabd6bb9dbbe9d0	State Available	Block Public Access Off	DNS hostnames Disabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-0354b00ba661b75e	Main route table rtb-0d909e3d347eb093d
Main network ACL acl-036e8c2ba94a4e92c	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group) -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 973629900070
Encryption control ID -	Encryption control mode -		

Resource map CIDRs Flow logs Tags Integrations

Resource map Info

Show all details

Step 2: Create Public and Private Subnets

- Go to Subnets → Create Subnet.
- Create Public Subnet:
 - CIDR: 10.0.1.0/24
 - Enable Auto-assign Public IP.
- Create Private Subnet:
 - CIDR: 10.0.2.0/24
 - Disable Auto-assign Public IP.

← → ↺ ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#subnets:v=3

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

VPC > Subnets

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

- Route tables
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

Select a subnet

Subnets (2) Info

Find subnets by attribute or tag

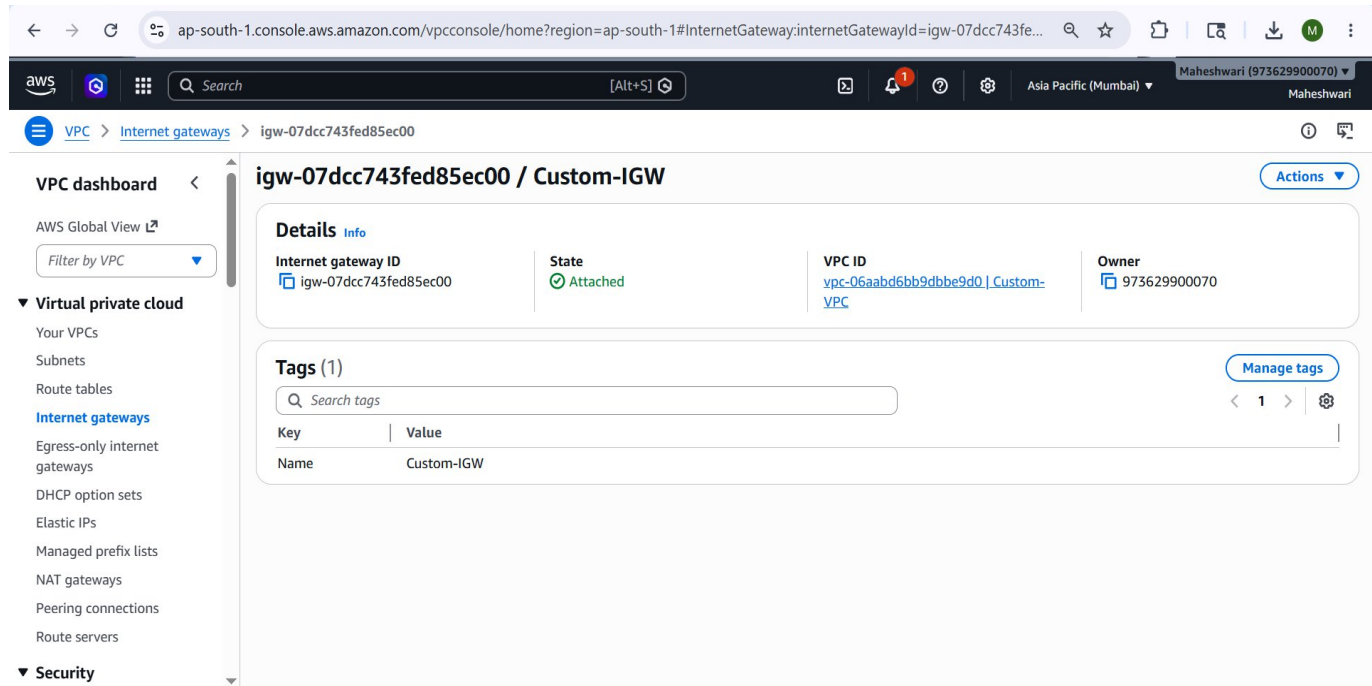
☐	Name	Subnet ID	State	VPC	Block Public...
☐	Public-Subnet	subnet-0b0fc58d19b4e9987	Available	vpc-06aabd6bb9dbbe9d0 Cus...	Off
☐	Private-Subnet	subnet-02a876c5a95b6455e	Available	vpc-06aabd6bb9dbbe9d0 Cus...	Off

Last updated about 8 hours ago

Actions Create subnet

Step 3: Create and Attach Internet Gateway

- Go to Internet Gateways → Create Internet Gateway.
- Name: Custom-IGW.
- Attach it to the Custom VPC.



The screenshot shows the AWS Management Console interface for an Internet Gateway. The breadcrumb navigation at the top indicates the path: VPC > Internet gateways > igw-07dcc743fed85ec00. The main header displays the gateway's name, "igw-07dcc743fed85ec00 / Custom-IGW", along with an "Actions" button. The "Details" section, which is currently expanded, shows the following information:

- Internet gateway ID:** igw-07dcc743fed85ec00
- State:** Attached (indicated by a green checkmark icon)
- VPC ID:** vpc-06aabd6bb9dbbe9d0 | Custom-VPC
- Owner:** 973629900070

Below the details, the "Tags (1)" section is visible, showing a single tag with the key "Name" and the value "Custom-IGW". The left-hand navigation menu includes sections for "Virtual private cloud" (containing links to VPCs, Subnets, Route tables, Internet gateways, Egress-only internet gateways, DHCP option sets, Elastic IPs, Managed prefix lists, NAT gateways, Peering connections, and Route servers) and "Security".

Step 4: Create NAT Gateway

- Allocate Elastic IP.
- Go to NAT Gateways → Create NAT Gateway.
- Select Public Subnet.
- Attach Elastic IP.
- Create NAT Gateway.

ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#NatGatewayDetails:natGatewayId=nat-18d40552b07...

Search [Alt+S]

Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

VPC > NAT gateways > nat-18d40552b0751a79c

VPC dashboard

AWS Global View

Filter by VPC

- Virtual private cloud
 - Your VPCs
 - Subnets
 - Route tables
 - Internet gateways
 - Egress-only internet gateways
 - DHCP option sets
 - Elastic IPs
 - Managed prefix lists
 - NAT gateways**
 - Peering connections
 - Route servers
- Security

nat-18d40552b0751a79c / Custom-NAT

Actions

Details

NAT gateway ID nat-18d40552b0751a79c	Availability mode Regional	State Available	State message -
NAT gateway ARN arn:aws:ec2:ap-south-1:97362990:0070:natgateway/nat-18d40552b0751a79c	Connectivity type Public	Created Saturday, May 23, 2026 at 12:02:28 GMT+5:30	Deleted -
VPC vpc-06aabd6bb9dbbe9d0 / Custom-VPC	Method of EIP allocation Automatic		

IP addresses Monitoring Flow logs Tags

Associated IP addresses (1)

Search

IP address	Status	Availability Zone	Allocation ID

Step 5: Configure Route Tables

- **Create Public Route Table:**
 - Add route 0.0.0.0/0 → Internet Gateway.
 - Associate with Public Subnet.
- **Create Private Route Table:**
 - Add route 0.0.0.0/0 → NAT Gateway.
 - Associate with Private Subnet.

ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#RouteTables:

Search [Alt+S]

Asia Pacific (Mumbai) Maheshwari (973629900070) Maheshwari

VPC > Route tables

VPC dashboard

AWS Global View

Filter by VPC

- Virtual private cloud
 - Your VPCs
 - Subnets
 - Route tables**
 - Internet gateways
 - Egress-only internet gateways
 - DHCP option sets
 - Elastic IPs
 - Managed prefix lists
 - NAT gateways
 - Peering connections
 - Route servers
- Security

Route tables (5)

Last updated 25 minutes ago

Find route tables by attribute or tag

	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	-	rtb-060a75de9e8a61c25	-	-	Yes	vpc-0bf18dc14
☐	-	rtb-071aecb0a6f1f6f91	-	nat-18d40552b07...	No	vpc-06aabd6bl
☐	Private-RT	rtb-059b92c0f2b6ce2f0	subnet-02a876c5a95b64...	-	No	vpc-06aabd6bl
☐	-	rtb-0d909e3d347eb093d	-	-	Yes	vpc-06aabd6bl
☐	Public-RT	rtb-0ff131d0e2b9092ad	subnet-0b0fc58d19b4e9...	-	No	vpc-06aabd6bl

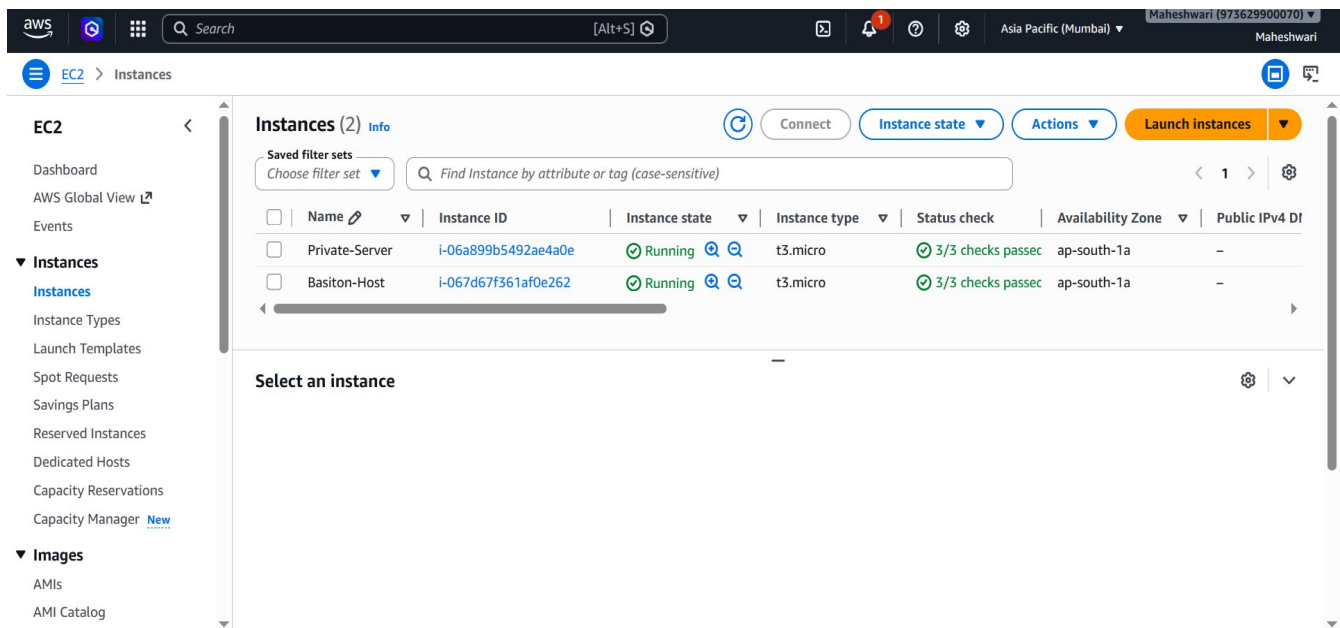
Select a route table

Step 6: Launch Bastion Host

- Go to EC2 → Launch Instance.
- Name: Bastion-Host.
- Choose Amazon Linux AMI.
- Select Public Subnet.
- Enable Auto-assign Public IP.
- Create Security Group:
Allow SSH (Port 22) from your IP.
- Launch with Key Pair.

Step 7: Launch Private EC2 Instance

- Go to EC2 → Launch Instance.
- Name: Private-Server.
- Choose Amazon Linux AMI.
- Select Private Subnet.
- Disable Public IP.
- Create Security Group:
 - Allow SSH only from Bastion Host Security Group.
- Launch with Key Pair.



Step 8: Connect Using Windows CMD (SSH Client)

- Add your private key to your local Windows SSH agent memory.
- SSH into the Bastion Host using Agent Forwarding to securely authenticate down the network path.
- `ssh -A ec2-user@<BASTION-PUBLIC-IP>` Once inside the Bastion session, jump directly into the Private Instance using its private IP address.
- `ssh ec2-user@<PRIVATE-INSTANCE-PRIVATE-IP>`

Step 9: Verify NAT Gateway Internet Access

- SSH into Private Instance through Bastion Host.
- Run:
sudo yum update -y
- Verify that internet access works successfully from the private subnet.

```
#
##### Amazon Linux 2023
#####
#####
##### \#/ https://aws.amazon.com/linux/amazon-linux-2023
##### V~' '->
#####
##### /m/'

Last login: Sat May 23 09:18:07 2026 from 13.233.177.3
[ec2-user@ip-10-0-1-214 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository                               315 kB/s | 38 kB    00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-1-214 ~]$ sudo yum httpd -y
No such command: httpd. Please use /usr/bin/yum --help
It could be a YUM plugin command, try: "yum install 'dnf-command(httpd)'"
[ec2-user@ip-10-0-1-214 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:02:52 ago on Sat May 23 09:24:45 2026.
Dependencies resolved.

=====
Package                                Architecture      Version                                Repository          Size
=====
Installing:
httpd                                   x86_64            2.4.66-1.amzn2023.0.1                amazonlinux          47 k
Installing dependencies:
apr                                     x86_64            1.7.5-1.amzn2023.0.4                amazonlinux          129 k
apr-util                               x86_64            1.6.3-1.amzn2023.0.2                amazonlinux          97 k
apr-util-ldap                           x86_64            1.6.3-1.amzn2023.0.2                amazonlinux          13 k
generic-logos-httpd                     noarch            18.0.0-12.amzn2023.0.3              amazonlinux          19 k
httpd-core                              x86_64            2.4.66-1.amzn2023.0.1                amazonlinux          1.4 M
httpd-filesystem                        noarch            2.4.66-1.amzn2023.0.1                amazonlinux          13 k
httpd-tools                             x86_64            2.4.66-1.amzn2023.0.1                amazonlinux          81 k
```

```
ec2-user@ip-10-0-1-214:~
Verifying      : httpd-2.4.66-1.amzn2023.0.1.x86_64                6/13
Verifying      : httpd-core-2.4.66-1.amzn2023.0.1.x86_64          7/13
Verifying      : httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch     8/13
Verifying      : httpd-tools-2.4.66-1.amzn2023.0.1.x86_64         9/13
Verifying      : libbrotli-1.0.9-4.amzn2023.0.2.x86_64           10/13
Verifying      : mailcap-2.1.49-3.amzn2023.0.3.noarch             11/13
Verifying      : mod_http2-2.0.27-1.amzn2023.0.3.x86_64           12/13
Verifying      : mod_lua-2.4.66-1.amzn2023.0.1.x86_64             13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64          apr-util-1.6.3-1.amzn2023.0.2.x86_64          apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  httpd-2.4.66-1.amzn2023.0.1.x86_64
httpd-core-2.4.66-1.amzn2023.0.1.x86_64      httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch  httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64      mailcap-2.1.49-3.amzn2023.0.3.noarch          mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-1-214 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-1-214 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-1-214 ~]$ sudo systemctl status httpd
* httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-05-23 09:28:13 UTC; 54s ago
     Docs: man:httpd.service(8)
   Main PID: 26366 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 1014)
   Memory: 13.5M
     CPU: 110ms
   CGroup: /system.slice/httpd.service
           └─26366 /usr/sbin/httpd -DFOREGROUND
             └─26368 /usr/sbin/httpd -DFOREGROUND
               └─26371 /usr/sbin/httpd -DFOREGROUND
                 └─26382 /usr/sbin/httpd -DFOREGROUND
                   └─26400 /usr/sbin/httpd -DFOREGROUND

May 23 09:28:13 ip-10-0-1-214.ap-south-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
May 23 09:28:13 ip-10-0-1-214.ap-south-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
```



```

ifyping      : libbrotli-1.0.9-4.amzn2023.0.2.x86_64      10/13
ifyping      : mailcap-2.1.49-3.amzn2023.0.3.noarch      11/13
ifyping      : mod_http2-2.0.27-1.amzn2023.0.3.x86_64    12/13
ifyping      : mod_lua-2.4.66-1.amzn2023.0.1.x86_64      13/13

lled:
-1.7.5-1.amzn2023.0.4.x86_64      apr-util-1.6.3-1.amzn2023.0.2.x86_64      apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
-util-openssl-1.6.3-1.amzn2023.0.2.x86_64      generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch      httpd-2.4.66-1.amzn2023.0.1.x86_64
pd-core-2.4.66-1.amzn2023.0.1.x86_64      httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch      httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
brotli-1.0.9-4.amzn2023.0.2.x86_64      mailcap-2.1.49-3.amzn2023.0.3.noarch      mod_http2-2.0.27-1.amzn2023.0.3.x86_64
_lua-2.4.66-1.amzn2023.0.1.x86_64

ete!
user@ip-10-0-1-214 ~]$ sudo systemctl start httpd
user@ip-10-0-1-214 ~]$ sudo systemctl enable httpd
ed symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
user@ip-10-0-1-214 ~]$ sudo systemctl status httpd
pd.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
Active: active (running) since Sat 2026-05-23 09:28:13 UTC; 54s ago
   Docs: man:httpd.service(8)
   Main PID: 26366 (httpd)
Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:  0 B/sec"
Tasks: 177 (limit: 1014)
Memory: 13.5M
CPU: 110ms
CGroup: /system.slice/httpd.service
└─26366 /usr/sbin/httpd -DFOREGROUND
  └─26368 /usr/sbin/httpd -DFOREGROUND
    └─26371 /usr/sbin/httpd -DFOREGROUND
      └─26382 /usr/sbin/httpd -DFOREGROUND
        └─26400 /usr/sbin/httpd -DFOREGROUND

B 09:28:13 ip-10-0-1-214.ap-south-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
B 09:28:13 ip-10-0-1-214.ap-south-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
B 09:28:13 ip-10-0-1-214.ap-south-1.compute.internal httpd[26366]: Server configured, listening on: port 80
user@ip-10-0-1-214 ~]$ q
: q: command not found
user@ip-10-0-1-214 ~]$ echo "<h1>My AWS Web Server is Working</h1>" | sudo tee /var/www/html/index.html
y AWS Web Server is Working</h1>
user@ip-10-0-1-214 ~]$

```



Troubleshooting:

Issue 1: Unable to SSH into Private Instance

Possible Causes:

- Security Group rules not configured properly.
- Bastion Host not accessible.
- Incorrect private key permissions.

Solution:

- Verify Security Group inbound rules.
- Ensure Bastion Host is in public subnet with public IP.
- Use correct private key and permissions.

Issue 2: Private Instance Cannot Access Internet

Possible Causes:

- NAT Gateway not configured.
- Incorrect Route Table association.
- Missing Internet Gateway route.

Solution:

- Verify NAT Gateway status is Available.
- Ensure private subnet route table points to NAT Gateway.
- Ensure public subnet route table points to Internet Gateway.

Issue 3: Connection Timeout from Bastion Host

Possible Causes:

- Private instance security group does not allow SSH from Bastion Host.
- Incorrect private IP address.

Solution:

- Allow Port 22 from Bastion Host Security Group.
- Verify private instance IP address.

Conclusion:

The Custom VPC Setup with Bastion Host and NAT Gateway project provides a secure and scalable cloud networking architecture in AWS. By separating resources into public and private subnets, the project improves security and minimizes direct exposure of private resources to the internet.

The Bastion Host ensures secure administrative access, while the NAT Gateway enables private instances to access the internet safely for updates and package installations. This architecture is highly suitable for production environments and follows AWS best practices for secure network design.